

Stationarity Tests

2026-04-17

Introduction

Stationarity — constant mean, variance, and autocovariance over time — is a fundamental prerequisite for most classical time-series models (ARIMA, VAR, spectral analysis). Non-stationary series typically need to be made stationary by differencing, detrending, or transformation before model fitting; forgetting this step produces nonsense estimates and invalid forecasts. Formal stationarity testing combines several tests with complementary nulls to give a robust verdict.

Prerequisites

A working understanding of time-series basics, autocorrelation, the difference between strict and weak stationarity, and the role of differencing in inducing stationarity.

Theory

The standard test battery comprises:

- **Augmented Dickey-Fuller (ADF)**: null = unit root (non-stationary), alternative = stationary or trend-stationary. Rejection supports stationarity.
- **KPSS**: null = stationarity (level or trend), alternative = unit root. Rejection supports non-stationarity.
- **Phillips-Perron (PP)**: non-parametric variant of ADF, robust to autocorrelation and heteroscedasticity in errors.

Pair them: stationary if ADF rejects *and* KPSS does not; non-stationary if ADF fails to reject *and* KPSS rejects. Disagreement (both reject, neither rejects) is ambiguous and may indicate near-unit-root behaviour, structural breaks, or fractional integration.

For series with structural breaks, standard tests are invalid; use Zivot-Andrews (`urca::ur.za()`) which extends the framework to allow one endogenous break.

Assumptions

Choice of test depends on the alternative — level vs trend stationarity for KPSS, drift vs trend specification for ADF. Adequate sample size for asymptotic critical values.

R Implementation

```
library(tseries); library(urca)

x <- AirPassengers

# ADF
adf.test(log(x))

# KPSS
kpss.test(log(x))
```

```
# Phillips-Perron
pp.test(log(x))

# Differencing to achieve stationarity
ndiffs(log(x)) # forecast::ndiffs
```

Output & Results

Each test returns a statistic and p -value. `forecast::ndiffs()` and `nsdiffs()` automate the differencing decision by running KPSS or other tests at each candidate differencing order until stationarity is supported.

Interpretation

A reporting sentence: “On the log-transformed AirPassengers series, ADF failed to reject the unit-root null ($p = 0.99$) and KPSS rejected level stationarity ($p < 0.05$); both agree the series is non-stationary. After first-differencing plus seasonal differencing (order $(0, 1, 0)(0, 1, 0)_{12}$), all three tests supported stationarity, justifying ARIMA modelling on the doubly-differenced series.” Always pair ADF with KPSS and report both verdicts.

Practical Tips

- Run ADF and KPSS together; their conclusions should agree, and disagreement is itself diagnostic information.
- For series with a clear trend, test trend-stationarity explicitly (`kpss.test(x, null = "Trend")` and `adf.test` with trend specification).
- Structural breaks invalidate standard tests; use Zivot-Andrews (`urca::ur.za()`) when a break is plausible.
- `forecast::ndiffs()` and `nsdiffs()` automate the choice of regular and seasonal differencing orders for ARIMA model preparation.
- Visual inspection of the series — plotting it and looking for trends, level shifts, or changing variance — usually suggests the right transformation before any test is run.
- For panel data (multiple time series), use Im-Pesaran-Shin or Levin-Lin-Chu panel unit-root tests; running univariate tests on each series ignores the panel structure.

R Packages Used

`tseries::adf.test()`, `kpss.test()`, `pp.test()` for the standard battery; `urca::ur.df()`, `ur.kpss()`, `ur.pp()`, `ur.za()` for richer output and structural-break extensions; `forecast::ndiffs()` and `nsdiffs()` for automated differencing decisions; `feasts::unitroot_kpss()` and `unitroot_ndiffs()` for the modern tidyverts interface.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Time series basics